



ENSTA PARIS
INSTITUT POLYTECHNIQUE DE PARIS

CSC_5RO01_TA

Projet

by

Gianluca BAGHINO, Guilherme TROFINO et Natalia GALLEGO

supervised by
Frédéric THOMAS
Vicent DAVID
Stéphane LOUISE

Confidentiality Notice
Non-confidential and publishable report

ROBOTIQUE
SCIENCES ET TECHNOLOGIES DE L'INFORMATION ET COMMUNICATION

Paris, France
7 avril 2025

Table des matières

1	Introduction	2
2	Exercice 1	3
2.1	Question 1	3
2.2	Question 2	4
2.3	Question 3	4
2.4	Question 4	4
2.5	Question 5	4
2.6	Question 6	5
2.6.1	Architecture logicielle	5
2.6.2	Classe <code>AcquisitionManager</code>	6
2.6.3	Classe <code>MessageAdder</code>	6
2.6.4	Diagramme de Séquence	7
2.7	Question 7	9
2.8	Question 8	10
2.9	Question 9	10
2.10	Question 10	10
2.10.1	Avantages POSIX	10
2.10.2	Désavantage	11
2.10.3	Conclusion	11
2.11	Question 11	11
2.12	Question 12	11
2.13	Question 13	13
2.14	Question 14	13
2.15	Question 15	15
2.16	Question 16	15
2.17	Question 17	15
2.17.1	Structure Globale	16
2.17.2	Séquence Temporelle	16
2.17.3	Garantie des Exigences	16
2.18	Question 18	16

1. Introduction

Dans le cadre de ce projet, une application multitâche sera conçue et mise en œuvre en environnement Linux, mettant en œuvre des mécanismes POSIX et C11 pour la gestion des threads et de la synchronisation. L'objectif principal est de développer un accumulateur multitâche, **MultitaskingAccumulator**, qui reçoit des données asynchrones provenant d'un **SensorManager**, les somme et envoie les résultats à un **Display** pour affichage.

Ce projet nous permettra d'explorer deux approches de conception de systèmes multitâches :

- Une approche dirigée par les événements, où les tâches réagissent à des signaux.
- Une approche dirigée par le temps, où les tâches doivent respecter un certain rythme d'exécution.

Enfin, la solution est structurée en respectant les consignes fournies, en veillant à une séparation claire entre les différents composants, et en assurant un affichage correct des résultats et diagnostics.

Les algorithmes pour ce projet se trouvent sur [GitHub](#).

2. Exercice 1

2.1. Question 1

L'architecture logicielle du `MultitaskingAccumulator` est complétée, en allouant correctement les exigences spécifiées aux différents composants du système. L'objectif est d'assurer une répartition cohérente des responsabilités entre les modules `AcquisitionManager`, `MessageAdder` et `DisplayManager`, tout en respectant les interactions définies.

Cette modélisation permet d'avoir une vision claire du fonctionnement global avant d'entrer dans l'implémentation.

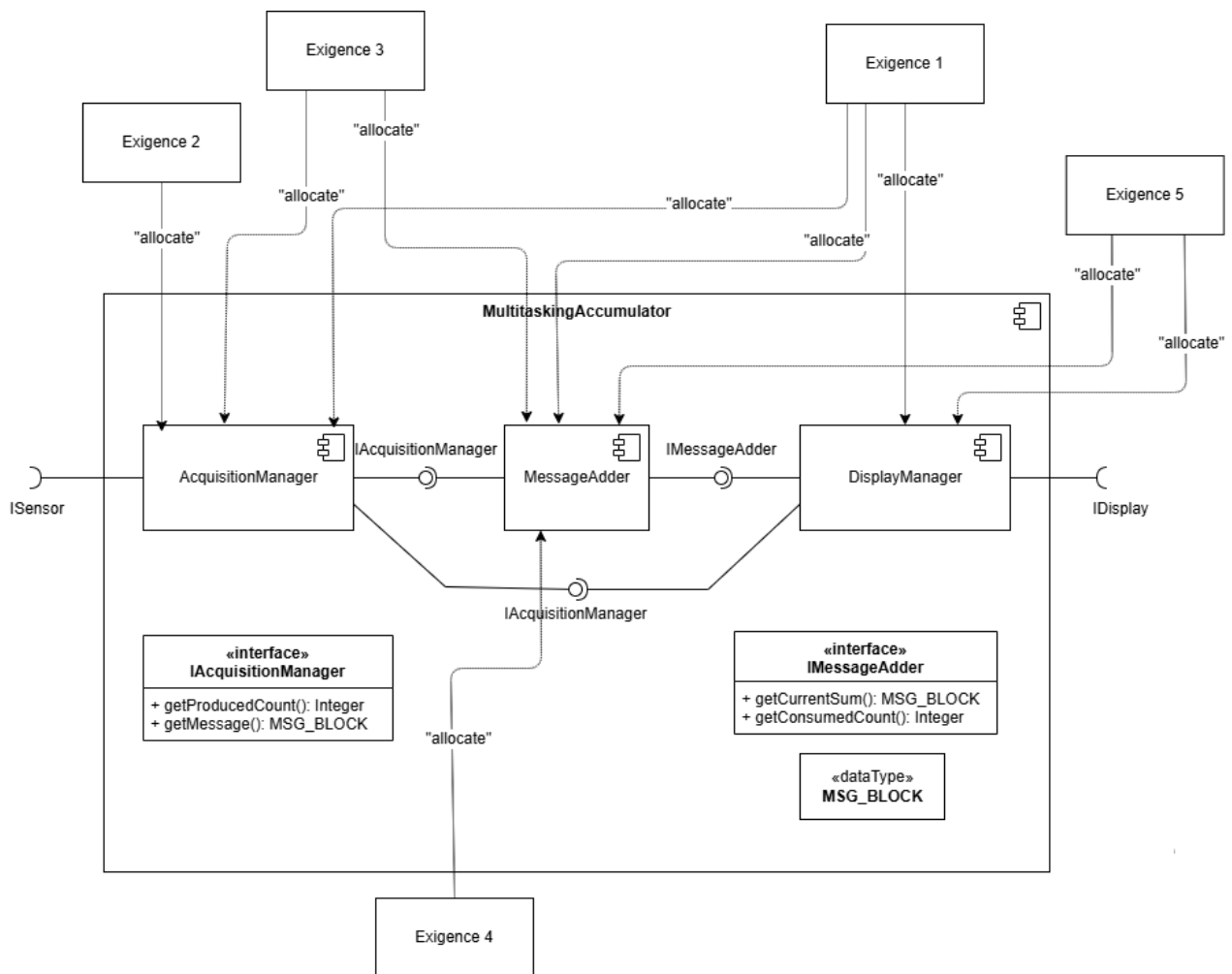


FIGURE 2.1 : Architecture Logicielle avec Allocation des Exigences.

2.2. Question 2

L'approche dirigée par les événements est asynchrone. Contrairement aux approches synchrones, qui se basent sur un calendrier fixe ou des événements temporels prédéfinis comme dans le modèle time-triggered, cette approche exécute les tâches dès qu'un événement survient, rendant le système réactif.

Dans ce programme, par exemple, les quatre threads du gestionnaire d'acquisition peuvent s'exécuter simultanément si le processeur dispose de plusieurs cœurs. Même dans un système à cœur unique, les threads sont gérés selon les événements ou blocages qui surviennent.

Cette méthode évite d'avoir une boucle principale bloquante et garantit un comportement réactif. En revanche, à la différence d'une approche time-triggered, l'asynchronisme est plus visible pour l'utilisateur, car celui-ci n'a pas à s'occuper de la gestion des moments précis de l'exécution des tâches.

2.3. Question 3

La méthode `messageCheck` implémente une stratégie de Sanity Checking, qui fait partie des Failure Detection. Cette méthode permet de vérifier l'intégrité des données dans un message, en s'assurant qu'il n'y a pas de corruption ou d'erreurs avant de le traiter.

Le Sanity Checking consiste à contrôler la cohérence logique et structurelle des données du message, vérifiant qu'elles sont conformes à ce qui est attendu et n'ont pas été altérées de manière imprévisible.

2.4. Question 4

À ce stade, le programme consiste en un seul processus principal qui est démarré en exécutant `Multitasking-Accumulator.c`. Ce processus contient plusieurs threads POSIX :

- 1 thread pour le processus principal (main)
- 4 threads pour les producteurs gérés par l'`AcquisitionManager` (un thread par capteur)
- 1 thread pour le `MessageAdder`, chargé de cumuler les données
- 1 thread pour le `DisplayManager`, responsable de l'affichage des messages

2.5. Question 5

La fonction `messageCheck()` vérifie l'intégrité d'un message en calculant son **checksum** à l'aide d'une opération XOR cumulative sur les données du message. Tout d'abord, elle initialise une variable `tcheck` à zéro, qui servira à stocker le checksum calculé. Ensuite, elle parcourt le tableau `mData` du message (`MSG_BLOCK`), en appliquant l'opération XOR (`^`) sur chacun de ses éléments.

Une fois ce processus terminé, elle compare la valeur obtenue avec le checksum stocké dans le message (`mBlock->checksum`). Si les deux valeurs correspondent, le message est considéré comme valide et un message de confirmation est affiché. Dans le cas contraire, un message d'erreur indique que le message est corrompu. Cette méthode permet de détecter d'éventuelles modifications accidentelles des données, garantissant ainsi leur intégrité lors de la transmission.

2.6. Question 6

2.6.1. Architecture logicielle

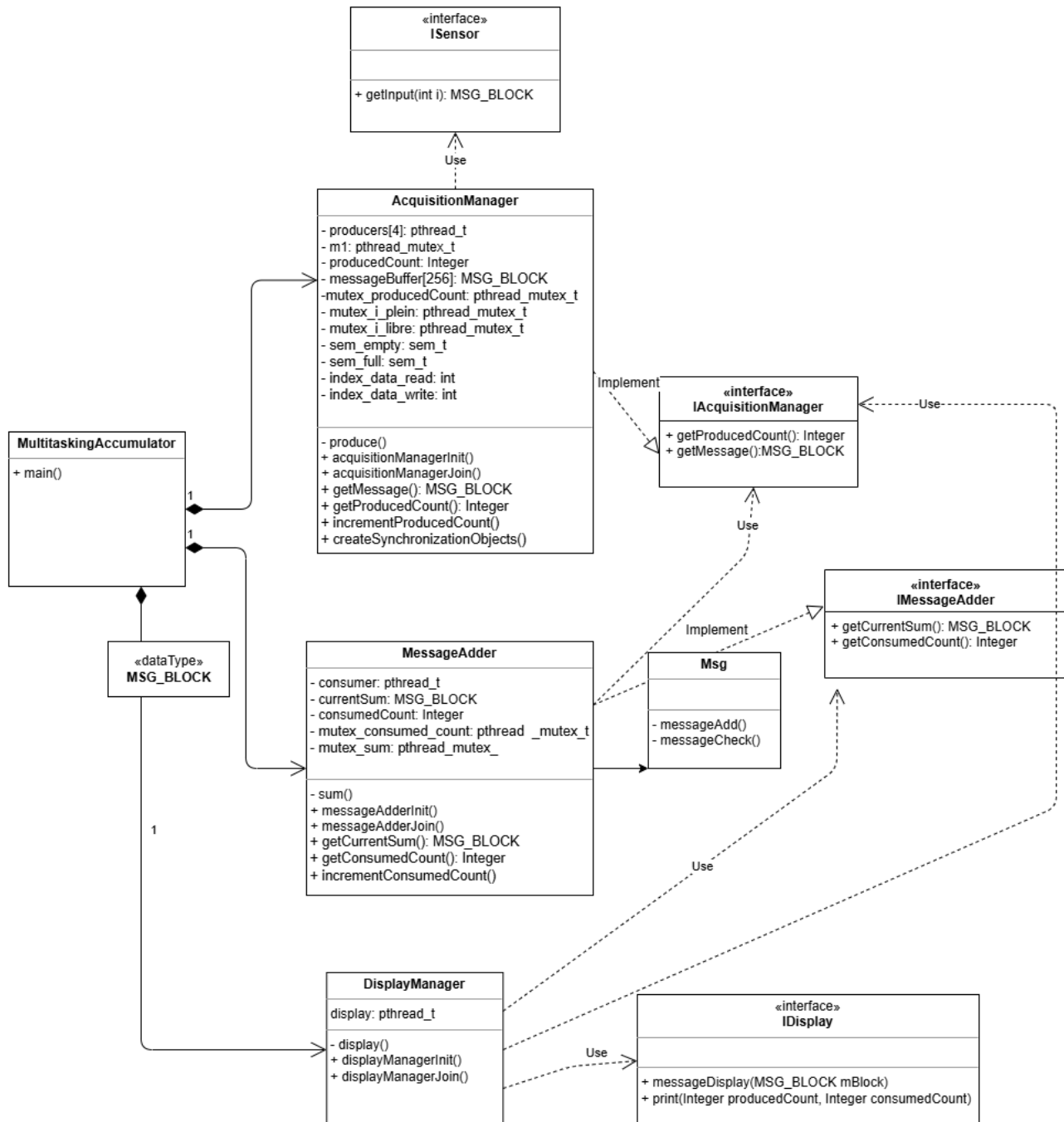


FIGURE 2.2 : Architecture Logicielle Détaillée de MultitaskingAccumulator.

2.6.2. Classe AcquisitionManager

1. Attributs :

- `producers[4]:pthread_t` : représente les threads qui gèreront la production de messages.
- `messageBuffer[256]:MSG_BLOCK` : buffer pour écriture et lecture des messages.
- `producedCount:int` : permet compter le nombre de messages produits.
- `mutex_producedCount:pthread_mutex_t` : garantir la sécurité des threads lors de l'accès.
- `mutex_i_plein, mutex_i_libre:pthread_mutex_t` : contrôler l'accès aux buffers.
- `sem_empty, sem_full:sem_t` : gérer l'espace du buffer.
- `index_data_read, index_data_write:int` : indices des données dans les buffers.

2. Méthodes :

- `getMessage()` : utilisée par le consommateur pour récupérer des messages du buffer.
- `AcquisitionManagerInit()` : initialise toutes les ressources.
- `AcquisitionManagerJoin()` : attend que tous les `AcquisitionManager` threads aient terminé.
- `getProducedCount()` : retourne le nombre de messages produits.
- `incrementProducedCount()` : incrémente le nombre de messages produits à chaque nouveau message.
- `createSynchronizationObjects()` : initialise les sémaphores et les mutex.

2.6.3. Classe MessageAdder

1. Attributs :

- `consumer:pthread_t` : hhread consommateur.
- `currentSum:MSG_BLOCK` : contient la somme actuelle des messages traités.
- `consumedCount:int` : permet de suivre le bilan de messages consommés
- `mutex_consumed_count:pthread_mutex_t` : garantit la sécurité des threads lors de la mise à jour.
- `mutex_sum:pthread_mutex_t` : synchronise l'accès à la somme actuelle.

2. Méthodes :

- `messageAdderInit()` : initialise les sémaphores, les mutex et les variables pour le `MessageAdder`.
- `messageAdderJoin()` : attend que le thread consommateur termine son travail.
- `getCurrentSum()` : récupère la somme actuelle.
- `getConsumedCount()` : retourne le nombre de messages consommés.
- `incrementConsumedCount()` : incrémente le nombre de messages consommés.

2.6.4. Diagramme de Séquence

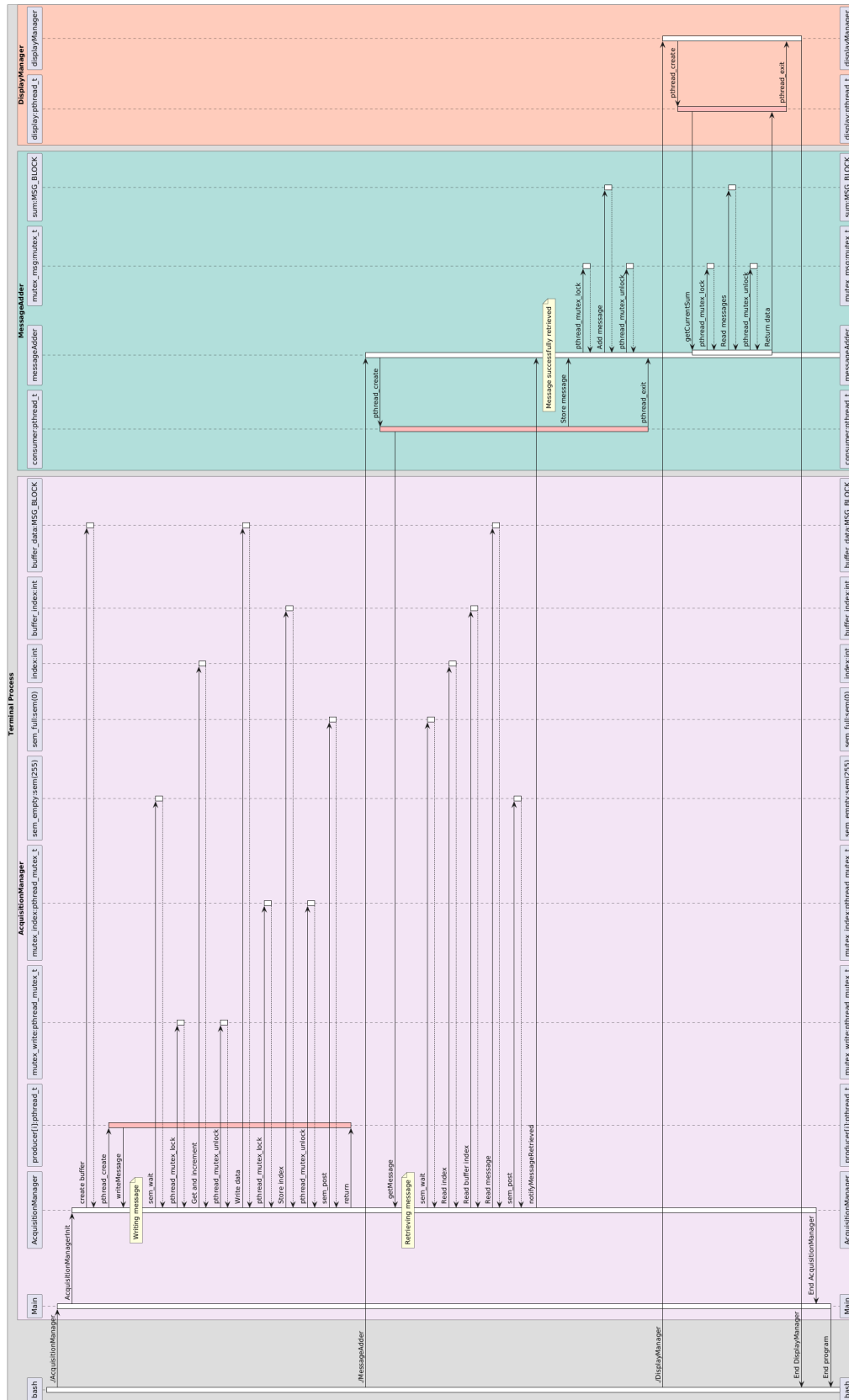


FIGURE 2.3 : Comportement dynamique de MultitaskingAccumulator

1. **bash** : démarre le programme.

- `bash->main:./AcquisitionManager` : lance `AcquisitionManager` class.
- `main->AcquisitionManager:AcquisitionManagerInit` : initialise `AcquisitionManager` objet.

2. **AcquisitionManager** : initialise le gestionnaire.

- `AcquisitionManager->buffer_data:create buffer` : crée un buffer.
- `AcquisitionManager->producer:pthread_create` : crée un thread producteur.

3. **AcquisitionManager** : produit et enregistre les messages.

- `producer->AcquisitionManager:writeMessage` : crée et envoie un message.
- `AcquisitionManager->sem_empty:sem_wait` : attend de l'espace.
- `AcquisitionManager->mutex_write:pthread_mutex_lock` : verrouille pour écrire.
- `AcquisitionManager->index:get and increment` : récupère et incrémente.
- `AcquisitionManager->buffer_data:write data` : écrit dans le buffer.
- `AcquisitionManager->mutex_index:pthread_mutex_lock` : verrouille pour l'index.
- `AcquisitionManager->buffer_index:store index` : stocke l'index.
- `AcquisitionManager->sem_full:sem_post` : signale un message.

4. **MessageAdder** : ajoute des messages.

- `bash->messageAdder:./MessageAdder` : lance `MessageAdder`.
- `messageAdder->consumer:pthread_create` : crée un thread consommateur.
- `consumer->AcquisitionManager:getMessage` : demande un message.
- `AcquisitionManager->sem_full:sem_wait` : attend un message.
- `AcquisitionManager->index:read index` : lit l'index.
- `AcquisitionManager->buffer_index:read buffer index` : lit l'emplacement.
- `AcquisitionManager->buffer_data:read message` : lit le message.
- `AcquisitionManager->sem_empty:sem_post` : signale de l'espace.
- `AcquisitionManager->messageAdder:notifyMessageRetrieved` : notifie.
- `consumer->messageAdder:store message` : stocke le message.

5. **MessageAdder** : ajoute à la somme.

- `messageAdder->mutex_msg:pthread_mutex_lock` : verrouille communication.
- `messageAdder->sum:add message` : ajoute une message.
- `messageAdder->mutex_msg:pthread_mutex_unlock` : déverrouille communication.
- `consumer->messageAdder:pthread_exit` : termine.

6. **DisplayManager** : affiche la somme.

- `bash->displayManager:./DisplayManager` : lance `DisplayManager`.
- `displayManager->display:pthread_create` : crée un thread display.
- `display->messageAdder:getCurrentSum` : demande la somme.
- `messageAdder->mutex_msg:pthread_mutex_lock` : verrouille.
- `messageAdder->sum:read messages` : lit la somme.
- `messageAdder->mutex_msg:pthread_mutex_unlock` : déverrouille.
- `messageAdder->display:return data` : renvoie la somme.
- `display->displayManager:pthread_exit`.
- `displayManager->bash:end DisplayManager`.
- `AcquisitionManager->main:end AcquisitionManager`.
- `main->bash:end program`.

2.7. Question 7

Voici le résultat de l'exécution :

```

1 [multitaskingAccumulator] Software initialization in progress...
2 [acquisitionManager] Synchronization initialization in progress...
3 [acquisitionManager] Synchronization initialization done.
4 [multitaskingAccumulator] Task initialization done.
5 [multitaskingAccumulator] Scheduling in progress...
6 [OK      ] Checksum validated
7 [OK      ] Checksum validated
8 [OK      ] Checksum validated
9 [OK      ] Checksum validated
10 [OK     ] Checksum validated
11 [OK     ] Checksum validated
12 Message
13 [displayManager] Produced messages: 3, Consumed messages: 1, Messages left: 2
14 [OK      ] Checksum validated
15 [acquisitionManager] 128111 termination
16 [OK      ] Checksum validated
17 [OK      ] Checksum validated
18 [acquisitionManager] 128110 termination
19 [OK      ] Checksum validated
20 [OK      ] Checksum validated
21 [acquisitionManager] 128109 termination
22 [OK      ] Checksum validated
23 [OK      ] Checksum validated
24 Message
25 [displayManager] Produced messages: 7, Consumed messages: 2, Messages left: 5
26 [OK      ] Checksum validated
27 [OK      ] Checksum validated
28 [OK      ] Checksum validated
29 [acquisitionManager] 128108 termination
30 [OK      ] Checksum validated
31 [OK      ] Checksum validated
32 Message
33 [displayManager] Produced messages: 8, Consumed messages: 4, Messages left: 4
34 [OK      ] Checksum validated
35 [OK      ] Checksum validated
36 [OK      ] Checksum validated
37 Message
38 [displayManager] Produced messages: 8, Consumed messages: 5, Messages left: 3
39 [OK      ] Checksum validated
40 [OK      ] Checksum validated
41 [OK      ] Checksum validated
42 [OK      ] Checksum validated
43 Message
44 [displayManager] Produced messages: 8, Consumed messages: 7, Messages left: 1
45 [OK      ] Checksum validated
46 [messageAdder] 128112 termination
47 [OK      ] Checksum validated
48 [OK      ] Checksum validated
49 Message
50 [displayManager] Produced messages: 8, Consumed messages: 8, Messages left: 0
51 [OK      ] Checksum validated
52 [OK      ] Checksum validated
53 Message

```

Listing 1 : Resultat Execution POSIX

2.8. Question 8

Pour garantir la cohérence entre la sortie diagnostic avec `print` et le cumul affiché `messageDisplay`, nous avons introduit une structure de données atomique et une logique de récupération synchronisée :

```
1 typedef struct MSG_BLOCK_TAG_ACCUMULATED
2 {
3     MSG_BLOCK message_block;
4     unsigned int consumed_count;
5 } MSG_BLOCK_ACCUMULATED;
```

Listing 2 : Modifications sur `msg.c`

```
1 static void *display(void *parameters)
2 {
3     D(printf("[displayManager] Thread created for display with id %d\n", getpid())); //gettid());
4     unsigned int difference_count = 0;
5
6     while(difference_count < DISPLAY_LOOP_LIMIT)
7     {
8         sleep(DISPLAY_SLEEP_TIME);
9         MSG_BLOCK_ACCUMULATED accumulated_count = getCurrentSum();
10        MSG_BLOCK current_count = accumulated_count.message_block;
11        unsigned int consumed_count = accumulated_count.consumed_count;
12
13        if (messageCheck(&current_count) == 0)
14            printf("[displayManager] Message corrupted\n");
15
16        messageDisplay(&current_count);
17
18        print(getProducedCount(), consumed_count);
19    }
20
21    printf("[displayManager] %d termination\n", getpid());
22    pthread_exit(NULL);
23 }
```

Listing 3 : Modifications sur `displayManager.c`

2.9. Question 9

Les variables déclarées volatile en C indiquent au compilateur que leur valeur peut être modifiée de manière imprévisible, en dehors du flux d'exécution normal du programme. Cela empêche le compilateur d'effectuer des optimisations potentiellement dangereuses comme la mise en cache dans un registre ou la suppression de lectures/écritures.

Dans une implémentation typique, cela est nécessaire dans les cas suivants :

1. **Accès matériel** : Une variable mappée sur un registre matériel (ex : capteurs, périphériques) dont la valeur change sans action explicite du code.
2. **Interruptions/Signaux** : Une variable modifiée dans une routine d'interruption (ISR) ou un gestionnaire de signal.
3. **Multithreading** : Variables partagées entre plusieurs threads (bien que volatile seul ne garantisse pas la synchronisation, il évite les optimisations locales).

Sans volatile, une boucle while (flag) ; pourrait être optimisée en while (true) ; si le compilateur ne détecte pas de modification de flag dans le code. Avec volatile, chaque accès à flag est forcé en mémoire

2.10. Question 10

2.10.1. Avantages POSIX

Caractéristiques intéressantes des processus POSIX pour la sûreté de fonctionnement :

1. **Isolation mémoire stricte** : Chaque processus possède son propre espace mémoire, ce qui empêche la corruption accidentelle ou malveillante de la mémoire d'autres processus. Cela limite la propagation de fautes.

2. **Fault containment** : Un crash d'un processus n'affecte pas directement les autres processus. Le système d'exploitation garantit le nettoyage des ressources en cas de terminaison.
3. **Séparation des privilèges** : Les processus peuvent s'exécuter avec des droits d'accès distincts, réduisant les risques de fuites ou d'abus de privilèges.
4. **Sécurité via mécanismes OS** : L'OS applique des politiques de sécurité, ce qui renforce la robustesse face aux attaques ou erreurs logicielles.
5. **Redémarrage indépendant** : Un processus défaillant peut être redémarré isolément sans impacter l'ensemble du système.

2.10.2. Désavantage

Caractéristiques non satisfaites par les processus POSIX :

1. **Surcharge de communication** : Les échanges inter-processus nécessitent des mécanismes explicites, introduisant une latence et une complexité accrues comparé aux threads.
2. **Coût en ressources** : La création/commutation de processus est plus lourde en CPU/mémoire que pour des threads, ce qui peut impacter les performances temps réel.
3. **Déterminisme limité** : L'ordonnancement par l'OS rend difficile la garantie de temps d'exécution strictement bornés, critique pour les systèmes temps réel sûrs.
4. **Partage de données complexe** : L'absence de mémoire partagée directe oblige à des mécanismes IPC synchronisés, augmentant les risques d'erreurs de conception.
5. **Gestion centralisée des erreurs** : Les erreurs globales ne sont pas toujours contenues, car l'OS reste un point unique de défaillance.

2.10.3. Conclusion

Les processus POSIX sont adaptés pour la tolérance aux fautes grâce à leur isolation, mais leur utilisation peut être contre-productive dans des systèmes nécessitant déterminisme, faible latence ou partage de données efficient. Une approche hybride (ex : processus + threads sécurisés) est souvent retenue en conception critique.

2.11. Question 11

Modifications proposées :

```
1 static void incrementProducedCount(void)
2     produce_count++;
3     ...
4 unsigned int getProducedCount(void) {
5     unsigned int p = 0;
6     p = produce_count;
7
8     return p;
9 }
```

Listing 4 : Modifications sur `acquisitionManagerAtomic.c`

2.12. Question 12

Modifications proposées :

```
1 static void incrementProducedCount(void)
2     produce_count++;
3     ...
4 unsigned int getProducedCount(void) {
5     unsigned int p = 0;
6     p = produce_count;
7
8     return p;
9 }
```

Listing 5 : Modifications sur `acquisitionManagerAtomic.c`

Voici le résultat de l'exécution :

```

1  [multitaskingAccumulator] Software initialization in progress...
2  [acquisitionManager] Synchronization initialization in progress...
3  [acquisitionManager] Synchronization initialization done.
4  [multitaskingAccumulator] Task initialization done.
5  [multitaskingAccumulator] Scheduling in progress...
6  [OK      ] Checksum validated
7  [OK      ] Checksum validated
8  [OK      ] Checksum validated
9  [OK      ] Checksum validated
10 [OK      ] Checksum validated
11 Message
12 [displayManager] Produced messages: 2, Consumed messages: 1, Messages left: 1
13 [OK      ] Checksum validated
14 [OK      ] Checksum validated
15 [acquisitionManager] 129405 termination
16 [OK      ] Checksum validated
17 [OK      ] Checksum validated
18 [OK      ] Checksum validated
19 [acquisitionManager] 129408 termination
20 [OK      ] Checksum validated
21 [acquisitionManager] 129407 termination
22 [OK      ] Checksum validated
23 [OK      ] Checksum validated
24 Message
25 [displayManager] Produced messages: 7, Consumed messages: 2, Messages left: 5
26 [OK      ] Checksum validated
27 [OK      ] Checksum validated
28 [acquisitionManager] 129406 termination
29 [OK      ] Checksum validated
30 [OK      ] Checksum validated
31 [OK      ] Checksum validated
32 Message
33 [displayManager] Produced messages: 8, Consumed messages: 4, Messages left: 4
34 [OK      ] Checksum validated
35 [OK      ] Checksum validated
36 [OK      ] Checksum validated
37 Message
38 [displayManager] Produced messages: 8, Consumed messages: 5, Messages left: 3
39 [OK      ] Checksum validated
40 [OK      ] Checksum validated
41 [OK      ] Checksum validated
42 [OK      ] Checksum validated
43 Message
44 [displayManager] Produced messages: 8, Consumed messages: 7, Messages left: 1
45 [OK      ] Checksum validated
46 [messageAdder] 129409 termination
47 [OK      ] Checksum validated
48 [OK      ] Checksum validated
49 Message
50 [displayManager] Produced messages: 8, Consumed messages: 8, Messages left: 0
51 [OK      ] Checksum validated
52 [OK      ] Checksum validated
53 Message

```

Listing 6 : Resultat Execution Atomic

2.13. Question 13

Modification proposées :

```
1 static void pCountLockTake(void)
2 {
3     int expected = 0;
4     while (!atomic_compare_exchange_weak(&lock, &expected, 1))
5         expected = 0;
6 }
7
8 static void pCountLockRelease(void)
9 {
10     lock = 0;
11 }
12
13 static void incrementProducedCount(void)
14 {
15     pCountLockTake();
16     produce_count++;
17     pCountLockRelease();
18 }
19
20 unsigned int getProducedCount(void)
21 {
22     unsigned int p = 0;
23
24     pCountLockTake();
25     p = produce_count;
26     pCountLockRelease();
27
28     return p;
29 }
```

Listing 7 : Modifications sur acquisitionManagerTestAndSet.c

2.14. Question 14

Modification proposées :

```
1 static void pCountLockTake(void)
2 {
3     int expected = 0;
4     while (!atomic_compare_exchange_weak(&lock, &expected, 1))
5         expected = 0;
6 }
7
8 static void pCountLockRelease(void)
9 {
10     lock = 0;
11 }
12
13 static void incrementProducedCount(void)
14 {
15     pCountLockTake();
16     produce_count++;
17     pCountLockRelease();
18 }
19
20 unsigned int getProducedCount(void)
21 {
22     unsigned int p = 0;
23
24     pCountLockTake();
25     p = produce_count;
26     pCountLockRelease();
27
28     return p;
29 }
```

Listing 8 : Modifications sur acquisitionManagerTestAndSet.c

Voici le résultat de l'exécution :

```

1 [multitaskingAccumulator] Software initialization in progress...
2 [acquisitionManager] Synchronization initialization in progress...
3 [acquisitionManager] Synchronization initialization done.
4 [multitaskingAccumulator] Task initialization done.
5 [multitaskingAccumulator] Scheduling in progress...
6 [OK      ] Checksum validated
7 [OK      ] Checksum validated
8 [OK      ] Checksum validated
9 [OK      ] Checksum validated
10 [OK     ] Checksum validated
11 [OK     ] Checksum validated
12 Message
13 [displayManager] Produced messages: 3, Consumed messages: 1, Messages left: 2
14 [OK      ] Checksum validated
15 [acquisitionManager] 129976 termination
16 [OK      ] Checksum validated
17 [OK      ] Checksum validated
18 [acquisitionManager] 129977 termination
19 [OK      ] Checksum validated
20 [OK      ] Checksum validated
21 [acquisitionManager] 129975 termination
22 [OK      ] Checksum validated
23 [OK      ] Checksum validated
24 Message
25 [displayManager] Produced messages: 7, Consumed messages: 2, Messages left: 5
26 [OK      ] Checksum validated
27 [acquisitionManager] 129974 termination
28 [OK      ] Checksum validated
29 [OK      ] Checksum validated
30 [OK      ] Checksum validated
31 [OK      ] Checksum validated
32 Message
33 [displayManager] Produced messages: 8, Consumed messages: 4, Messages left: 4
34 [OK      ] Checksum validated
35 [OK      ] Checksum validated
36 [OK      ] Checksum validated
37 Message
38 [displayManager] Produced messages: 8, Consumed messages: 5, Messages left: 3
39 [OK      ] Checksum validated
40 [OK      ] Checksum validated
41 [OK      ] Checksum validated
42 [OK      ] Checksum validated
43 Message
44 [displayManager] Produced messages: 8, Consumed messages: 7, Messages left: 1
45 [OK      ] Checksum validated
46 [messageAdder] 129978 termination
47 [OK      ] Checksum validated
48 [OK      ] Checksum validated
49 Message
50 [displayManager] Produced messages: 8, Consumed messages: 8, Messages left: 0
51 [OK      ] Checksum validated
52 [OK      ] Checksum validated
53 Message

```

Listing 9 : Resultat Execution POSIX

2.15. Question 15

On peut noter que l'exécution la plus efficace en termes de durée moyenne d'exécution est la configuration **Atomic**, car elle présente le temps d'exécution le plus faible avec une faible variance.

On observe que les configurations **POSIX** et **Test-and-Set** affichent des durées plus élevées, en raison de différents facteurs. Dans le cas de **POSIX**, afin d'éviter les conflits matériels et les erreurs d'information, l'utilisation d'un mutex interdit à plus d'un processus à la fois d'accéder aux ressources, ce qui entraîne une pause pour les autres processus et, par conséquent, un retard dans l'exécution du système.

Dans le cas de **Test-and-Set**, pour éviter également les conflits matériels et les erreurs d'information, l'algorithme met en place une "attente" en exécutant une implémentation simple d'un timeout. Cela empêche le système de poursuivre son exécution tant que cette boucle n'est pas interrompue, ce qui, comme pour **POSIX**, engendre des retards dans l'exécution du système.

Contrairement aux deux méthodes précédemment expliquées, l'implémentation atomique utilise le mot-clé **volatile**, natif de C++, qui désactive les optimisations du compilateur et permet des accès "simultanés" à la variable. Cela permet une réduction du temps d'exécution et, par conséquent, une augmentation de l'efficacité du système.

Cependant, il convient de souligner que la solution **Atomic** n'est pas recommandée, même si elle est valide dans cette application. En désactivant les vérifications de compilation, la garantie du bon fonctionnement du code n'est plus assurée, car il n'y a pas de gestion formelle des ressources ni des variables, ce qui peut conduire à des résultats inattendus et imprévisibles. Cela n'est donc pas recommandé pour un système embarqué en temps réel.

2.16. Question 16

Une approche dirigée par le temps **time-driven** est généralement considérée comme **synchrone** car :

1. Synchronisme :

- Les systèmes synchrones fonctionnent selon un cadencement temporel fixe comme horloge, intervalles prédéfinis.
- Les actions sont déclenchées ou coordonnées à des moments précis, indépendamment des événements externes.

2. Asynchronisme :

- Les systèmes asynchrones réagissent à des *événements ou signaux externes* sans dépendre d'un temps fixe.

3. Approche dirigée par le temps :

- Elle repose sur des cycles, horloges ou intervalles prédéterminés comme mise à jour périodique de données.
- Cette régularité temporelle implique une coordination synchrone, même si les traitements ultérieurs peuvent être asynchrones comme une tâche planifiée déclenche un processus en arrière-plan.

Bien que certaines phases puissent être asynchrones, l'élément déclencheur temporel, comme une horloge ou un timer, relève du synchronisme. Ainsi, une approche dirigée par le temps est principalement synchrone.

2.17. Question 17

Pour répondre aux exigences temporelles du système **MultitaskingAccumulator**, nous proposons une conception basée sur le Logical Execution Model de PsyC. Cette modélisation garantit une exécution prédictible et respecte les contraintes de temps réel.

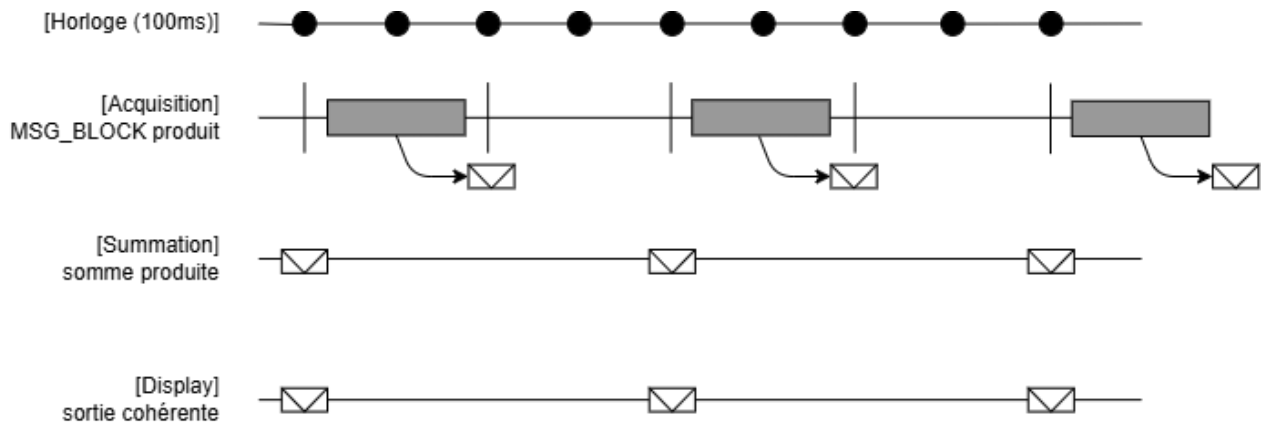


FIGURE 2.4 : Diagramme Temporelle.

2.17.1. Structure Globale

1. **Ligne d'Horloge** : une ligne horizontale représente l'horloge système, avec des ticks (points noirs) espacés de 100ms. Ces ticks servent de repères temporels pour synchroniser les tâches.
2. **Tâches** : Trois lignes horizontales représentent les tâches principales :
 - Acquisition : Lit les entrées des capteurs toutes les 100ms.
 - Summation : Cumule les données immédiatement après leur acquisition.
 - Display : Affiche le résultat dans un délai maximal de 600ms.
3. **Synchronisation** : Des lignes verticales (instructions advance en PsyC) alignées sur les ticks d'horloge synchronisent l'exécution des tâches.
4. **Messages** : Des enveloppes indiquent quand un MSG_BLOCK est produit et visible par les tâches consommatrices.

2.17.2. Séquence Temporelle

À chaque tick d'horloge, la tâche Acquisition lit une entrée et produit un MSG_BLOCK. La tâche Summation traite ce message dès sa disponibilité, sans attendre le tick suivant.

La tâche Display affiche la somme et les diagnostics au plus tard 6 ticks (600ms) après l'acquisition.

2.17.3. Garantie des Exigences

- **Exigence 6** (Acquisition périodique) : Les ticks forcent une exécution toutes les 100ms.
- **Exigence 7** (Latence inférieur à 600ms) : Le chemin critique (Acquisition, Summation, Display) est limité à 6 ticks, assurant un délai maximal de 600ms.

2.18. Question 18

Pour limiter le jitter (variabilité temporelle) des sorties, nous avons utilisé les instructions before de PsyC, représentées par des lignes verticales hachurées sur le diagramme temporel. Voici la démarche :

1. Les lignes hachurées indiquent un deadline avant le prochain point de synchronisation (advance).

Si une tâche doit s'exécuter avant le tick $T+100ms$, une ligne hachurée est placée à $T+100ms$.